TECHNICAL CONCEPT PAPER

stm8-rs: Rust for STM8 via SDCC

Feasibility, compiler architecture, restricted runtime, migration strategy, and validation roadmap

Version 1.0 | 2 November 2026 | F0 baseline specification

Executive summary

A limited but useful Rust firmware toolchain for selected STM8 microcontrollers is technically plausible. The strongest approach is to preserve the existing rustc frontend (including type checking, borrow checking, traits, generics, and monomorphization) while translating lowered Rust operations into C accepted by SDCC, whose STM8 machine-code backend is mature and maintained. The proposed target is not “a new Rust-like language,” but a constrained no_std execution environment and explicitly supported subset of Rust runtime behavior.

This is not a small compiler plugin. Core obstacles include custom target data layout, MIR/codegen integration, pointer and address-space semantics, an SDCC-compatible ABI, codegen correctness across C undefined-behavior rules, compiler intrinsics, and testing. A credible initial outcome is mixed C/Rust firmware, followed by standalone Rust application logic and selected drivers; broad core/crate support is a later milestone.

Decision thesis: lead STM8S103F3P6 and initial compatibility STM8S003F3P6 are the selected exact MCU part numbers. Confirm boards by marking and BOM; qualify only after passing the corresponding gates F0-F2. Prioritize SPL example functional coverage and code-size/performance parity before broad feature expansion.

Project mission — memory safety without hardware compromise: bring Rust’s modern type system, ownership model and compile-time correctness guarantees to resource-constrained 8-bit STM8 microcontrollers while preserving the code size, memory consumption and execution efficiency of traditional C firmware.

The objective is not simply to run Rust on STM8: it is to demonstrate that safer embedded software does not inherently require more powerful or expensive hardware. For a useful subset of STM8 peripherals, require compile-time rejection of invalid peripheral configurations and unsafe resource use, runtime correctness under real hardware conditions, and flash/RAM/stack/timing parity with equivalent SPL C firmware. By shifting correctness checks from runtime to compile time, specializing code for known hardware configurations and eliminating unnecessary abstractions, the project aims to make firmware safer by construction without making it more expensive to execute. This addresses runtime resource cost, not compiler implementation effort, and requires measurement rather than assuming every abstraction is free.

1. Background and motivation

1.1 Why AVR and STM8 are difficult compiler targets

AVR and STM8 are 8-bit microcontroller architectures with tight RAM/flash budgets, nonuniform registers, unusual addressing constraints, and strong hardware-specific ABI requirements. Generic compiler optimizations do not automatically produce efficient code on these targets. Multi-byte arithmetic, register allocation, stack discipline, interrupt entry, and separation of memory regions must be implemented with care.

LLVM support for AVR exists, but the Rust AVR target has historically been Tier 3. Tier 3 is a Rust project support designation, not an LLVM quality rating; it says that official testing/build guarantees are limited. It does not prove widespread wrong-code bugs. Older backend defects and limitations merit risk-based validation, rather than extrapolating a miscompilation rate from anecdotes.

1.2 STM8 toolchain situation

STM8 lacks an established upstream LLVM backend and an official rustc target. Existing practical STM8 compilers include commercial offerings and SDCC. SDCC is not merely an unfinished fallback: its STM8 port is categorized as mature in the SDCC release process, and SDCC 4.6.0 was released on 22 June 2026. Its optimizer and backend can be reused instead of recreating STM8 instruction selection, register allocation, assembly, and linking from scratch. [17][18]

This proposal assumes no existing functional STM8 Rust compiler. Existing STM32 Rust peripheral crates operate above a pre-existing ARM backend; equivalent STM8 peripheral descriptions alone would not solve code generation.

1.3 Project objective

Enable engineers to write and incrementally migrate simple real STM8 firmware in idiomatic Rust, using SDCC for target code generation. Use ownership, borrowing, enums, traits, const generics and type-state patterns to encode peripheral configurations, pin modes, protocol states, buffer extents and hardware invariants at compile time. Prefer abstractions whose checks are discharged during type checking, constant evaluation and monomorphization, leaving minimal or no runtime machinery.

Start with the STM8S103F3P6 bring-up device and then validate the STM8S003F3P6 compatibility device. Prefer safe application logic with hardware operations encapsulated in audited unsafe boundaries; neither entire product groups nor unvalidated packages are claimed supported. Zero-runtime-cost abstraction and C-compatible performance are validation targets, not assumptions: evaluate generated assembly and whole-firmware flash, RAM, stack and cycle measurements against equivalent SDCC C.

Project identity: stm8-rs is the project and intended public repository name. The compiler backend is named rustc_codegen_stm8_sdcc and the initial target specification is stm8-sdcc-medium.json. Later PAC/HAL package names such as stm8-pac and stm8-hal are proposals subject to availability checks at publication. The project name identifies this implementation and does not imply a pre-existing organization or working toolchain.

1.4 Pragmatism over abstraction

Zero-cost abstraction is a goal, not a dogma. Embedded software engineering requires explicit, measurable trade-offs under strict resource constraints. On the 8 KB flash / 1 KB RAM targets, prefer standard Rust abstractions when they provide safety and clarity at acceptable cost. When their overhead is prohibitive, favor smaller hardware-specific implementations with clearly documented limitations. A sophisticated generic HAL is not automatically better than a compact specialized driver.

Use a three-level implementation strategy:

1. Preferred: standard Rust types and abstractions that compile to efficient machine code while preserving the required safety and behavioral contracts.
2. Alternative: specialized, compile-time-configured abstractions that provide equivalent safety guarantees with lower or more predictable resource use.
3. Fallback: minimal hardware-specific code, including carefully encapsulated unsafe operations where necessary. Unsafe implementation is acceptable only behind an audited interface with explicit invariants and tests.

All three are valid outcomes when firmware satisfies its correctness and resource requirements. Generality, elegance, ergonomics and API completeness are optional; correctness, predictable resource usage and explicit safety contracts are not. If a correct implementation cannot fit the target, reduce the supported functionality rather than weakening memory safety or documented behavior.

For example, a tiny decimal-to-ASCII routine for UART debugging may be preferable to a generic allocation-free formatting framework, and a minimal safe driver with a narrowly scoped unsafe implementation may be preferable to a complex peripheral-ownership hierarchy. These are acceptable engineering compromises when measurements support them and their limitations are explicit.

This also answers why not simply continue using C and SDCC: the goal is to retain C's hardware efficiency while gaining Rust's compile-time correctness guarantees wherever they provide meaningful value—and to measure whether that goal has actually been achieved.

2. Recommended compiler architecture

```
Rust source (#![no_std])
        |
        v
rustc: parse / expand / type-check / borrow-check
        |
        v
Typed, monomorphized Rust operations (MIR / codegen integration)
        |
        v
[NEW] STM8-specific lowering and C emission
        |
        v
SDCC-compatible generated C + thin runtime/shims
        |
        v
SDCC STM8 backend, assembler, linker
```

```
        |
        v
Intel HEX / firmware image
```

The goal is not to reimplement Rust grammar, name resolution, macro expansion, trait solving, or borrowing. rustc provides that. A backend or MIR-based compiler integration provides target-specific lowering and emits C that SDCC can compile. Rust offers an unstable dynamically loadable codegen backend interface; the rustc development guide also cautions that backend-independent code still contains LLVM-related assumptions. A stand-alone rustc_driver/rustc_interface prototype is therefore appropriate before choosing a permanent integration layer. [13][19][20]

2.1 Architecture options and trade-offs

Option	Benefit	Main weakness
MIR / rustc → C → SDCC	Preserves real Rust checks and language semantics; reuses SDCC backend	Complex and unstable rustc integration; semantic lowering
Custom rustc codegen backend → C	Cleaner long-term interface if viable	Codegen traits are broad and version-sensitive
LLVM IR → C → SDCC	May reuse LLVM lowering	Address-space/ABI mismatch; lost semantics; optimizer interactions
syn-based Rust → C translator	Quick to prototype limited syntax	Lacks real type/borrow checking; scales badly
Native STM8 LLVM backend	Conventional Rust pipeline eventually	Large independent compiler engineering project

Recommendation: use rustc + MIR as the core design. A small C ABI shim is acceptable for startup, interrupts, and low-level system integration; it should not become a hidden reimplementation of the application.

2.2 Related implementations and reusable lessons

AVR is a primary reference for 8-bit target integration, while rustc_codegen_c is the closest architectural reference for MIR→C lowering. The current avr-none target uses LLVM's AVR backend for machine-code generation and AVR-GCC for linking and selected libgcc helpers; it does not translate MIR into C. The proposed STM8 route instead emits C for SDCC, which performs instruction selection and target code generation. [23][24][28]

- AVR target integration: study its 16-bit target specification, build-std=core workflow, libgcc inventory, target-cpu consistency, release-build guidance, and simulator/hardware tests. Reuse integration patterns and semantic tests, not LLVM instruction selection, AVR ABI, address spaces or assembly. Tier 3 status does not erase the implementation evidence, but it provides no official automatic build/test guarantee.
- Embedded ecosystem: study Rahix/avr-hal for PAC/HAL organization and rust-embedded/critical-section for the portable API and restore-state pattern. An STM8 implementation must preserve CC

and interrupt priorities independently. The API alone does not make shared-port read-modify-write safe. [25][26]

- Memory and validation: AVR and STM8 share 8-bit resource constraints but differ in byte order, address spaces, ABI and backend behavior. STM8 uses Harvard buses with a unified logical address space, unlike classic AVR's more explicit program/data distinction. Cross-architecture semantic comparisons are useful; instruction output and resource metrics are not STM8 pass/fail oracles.
- MIR→C implementation: evaluate rustc_codegen_c for MIR lowering, layout, intrinsics, validity and test organization. Its host-C output and claims do not establish STM8/SDCC correctness. Pin and review its revision and license before code reuse. The historical AVR-Rust Guidebook is useful background; current rustc target sources and schemas are authoritative. [27][28]

3. What "main Rust features" means

A restricted embedded target can still preserve a large part of the language. Distinguish compile-time language capabilities from runtime/library features. The following is a planned support matrix, not a feature list of an existing toolchain.

Feature group	Target scope	Key dependency
Ownership, borrowing, lifetimes	Preserve	Working rustc frontend
Structs, tuples, arrays, enums, match	Preserve	Correct layout and discriminant lowering
Traits, generics, impl blocks	Preserve with static dispatch first	Monomorphization and symbol emission
Option / Result	Preserve	Enum layout and core integration
Closures and deterministic Drop	Initial after validated basic MIR lowering	Captured environments, drop glue and cleanup paths
dyn Trait / trait objects	Deferred	Vtables and trait-object fat pointers
async / await	Later, selective	State-machine lowering; stringent RAM analysis
Heap allocation and unwinding	Out of first scope	Allocator and exception runtime
Threading / general atomics	Out of first scope	Hardware, critical sections, runtime support
bool, unit, 16-bit usize/isize	Initial	Target layouts, indexing, validity
Slice references &[T] / &mut [T]	Initial, validated	Pointer/length metadata representation and bounds checks
Function pointers and const fn	Initial, validated	ABI and compile-time evaluation
Ordinary statics; static mut	Initial / unsafe-only	Linkage and interrupt sharing
char	Deferred	Unicode scalar semantics when

		supported; never redefine as ASCII
&'static [u8; N]	Preferred initial after storage/layout validation	Thin pointer, compile-time length and immutable flash placement
UTF-8 string literals, &str / &'static str	Initial after storage/layout validation	Byte storage, pointer/length metadata, flash placement and UTF-8 validity
C string literals, &CStr	Initial after storage/ABI validation	NUL-terminated bytes, current metadata representation, c_char and C pointer ABI
StaticString<N> (experimental)	Deferred pending measured benefit	Checked const construction, static/lifetime proof and whole-image optimization
u64/i64; f32/f64	Deferred	Helper sizing, semantics and reentrancy
u128/i128; allocation, unwinding	Excluded initially	Not part of restricted runtime
Recursion and inline assembly	Deferred	Static stack proof / backend-specific contract
Unions, packed/aligned layouts	Deferred	C compatibility, alignment and aliasing

“no_std” does not mean “no core.” Producing a working core library for an unsupported target is a serious milestone. It requires target definitions and compiler-builtins, and may expose additional target assumptions. The first proof of concept may compile freestanding functions and a tiny hand-selected API before normal core-based firmware becomes viable.

4. Seven central engineering challenges

4.1 STM8 memory model and pointer semantics

Begin with selected STM8S103F3P6 and STM8S003F3P6 parts. SDCC --model-medium uses 16-bit object and function addresses; --model-large uses 16-bit object addresses and 24-bit function addresses. Initial compiler experiments use --model-medium exclusively; test and assert byte layout, big-endian multi-byte integer storage, function representation, alignment, and the __sdcccall(1) ABI. Do not mix memory models. Derive supported atomics conservatively from ISA and interrupt model; verify those assumptions experimentally.

Verify the SDCC medium-memory model and __sdcccall(1) register, stack, parameter and hidden-structure-return rules using emitted assembly, object files, reciprocal C/Rust calls, and hardware execution. These conventions are contracts, not assumptions inferred from host Rust.

```
unsafe fn write_gpio(addr: *mut u8, value: u8) {
```

```
    core::ptr::write_volatile(addr, value);
}
```

Generated C must preserve volatile side effects, ordering requirements, and aliasing guarantees without silently promoting pointers into an incompatible address space. Explicit safe wrappers for peripheral registers should be built only after primitive accesses are validated.

4.2 Validity, uninitialized storage, and C aliasing

Track MaybeUninit, padding, initializedness and reads of invalid values; bool/char/reference validity; enum discriminants, niche layouts, pointer provenance, and C effective-type/aliasing restrictions. A naïve C struct or cast may be invalid even if its byte size matches. Generated C must not introduce C undefined behavior for valid Rust inputs; unsupported operations must be rejected with diagnostics.

4.3 Rust arithmetic vs. C undefined behavior

Preserve ordinary Rust arithmetic in both overflow-checks enabled and disabled builds, wrapping_* and saturating_* where supported, shifts and out-of-range shift counts, truncation/casts, division and remainder, and signed MIN / -1; the latter must be handled even when overflow checks are disabled. Never lower signed wrapping addition into overflowing C signed arithmetic. Test u8/i8/u16/i16/u32/i32 boundary cases on target.

4.4 Layout, ABI, and interoperability

Rust repr(Rust) is not equivalent to C struct layout; enums can have niche representations. Generated C must use a computed target layout or explicit storage/offset operations. For #[repr(C)] and extern "C", compatibility means SDCC calling conventions, argument/return passing, bitfields, pointer classes, struct alignment, and symbol linkage—not just matching spelling. Treat an ABI test suite as a release prerequisite.

4.5 Stack usage, reentrancy, and interrupts

STM8 stack budgets are small. Audit every SDCC arithmetic/runtime helper emitted or linked into firmware, choose the appropriate reentrant library configuration (including --int-long-reent where applicable), and test helper calls from main and ISR contexts. Implement panic-in-ISR policy (abort or reset) and test it. On STM8, SDCC __critical has historically used sim/rim; rim sets CC I1:I0 priority flags rather than restoring the previous flags. Therefore a nested/ISR-safe Rust critical-section mechanism must not blindly use __critical. Prototype an audited shim that saves CC, masks interrupts and restores the original CC, including condition flags; confirm actual generated code in pinned SDCC and test nested calls, priority levels, early returns and ISR transitions. See SDCC manual §3.9.1, PM0044 and SDCC bug #3500.

4.6 core, intrinsics, and panic support

Investigate core during the earliest feasibility spike, not only beta: define a custom STM8 target JSON (endian, pointer width, integer alignment, atomic restrictions); pin nightly rustc/Cargo and rust-src; attempt cargo -Z build-std=core; enumerate failing compiler intrinsics, compiler-builtins, backend assumptions, and target-layout errors. Full core support need not be completed during the spike, but

its blockers must be inventoried. For later support, specify core::ptr, slices, bounds checks, memory helpers and panic=abort with a minimal handler. Reject unsupported operations explicitly.

AVR provides a concrete precedent for building upstream core on an 8-bit target with 16-bit pointers and C int using Cargo build-std. Its target specification also demonstrates explicit runtime-helper dependencies and consistent per-device CPU selection. This reduces target-integration uncertainty, but does not demonstrate that a MIR→C backend can compile core or preserve its semantics through SDCC. AVR's Tier 3 limitations still apply. [23][24]

4.7 String literals, C strings, and static storage

Prefer standard Rust representations and general backend optimizations over a specialized string subsystem. Use &'static [u8; N] for fixed LCD messages, protocol identifiers, lookup tables and calibration data; &[u8] for variable-length binary data; &str for valid UTF-8; &CStr for genuine NUL-terminated C interfaces; and [u8; N] with checked length management for mutable buffers. A fixed array reference has no runtime length metadata under the conventional model, while its N is known at compile time. Subject to layout validation, it is a thin 16-bit pointer with no terminator and no scan. [11][29][32]

Const-generic consumers can preserve N at compile time, but each distinct N may cause separate monomorphized code and increase total flash use. The MIR-to-C backend should optimize ordinary array references, compile-time indices and lengths, flash-resident constants and bounded loops generally, rather than special-casing strings. It must preserve required bounds checks and panic behavior and must not substitute strlen/strcmp or unbounded %s for length-based Rust strings.

Native c"..." literals remain the standard C-string baseline. CStr does not require UTF-8 and may hold arbitrary nonzero display-controller bytes; Unicode literal characters are encoded as UTF-8. Use to_bytes()/to_bytes_with_nul() for bounded Rust access and as_ptr() for read-only C calls. The reference representation and constant-time behavior of CStr operations are not permanent guarantees and must be checked against the pinned compiler. alloc::String and format! remain excluded initially.

StaticString<N> is an experimental dual-view abstraction, not a replacement for the standard types. It stores a private bytes: &'static [u8; N] where N includes exactly one trailing NUL. A checked const fn new() rejects N == 0, every interior NUL and a missing terminator; constant initializers fail at compile time, while runtime calls validate dynamically. Define LEN = N.saturating_sub(1) as a defensive adjustment: direct LEN access cannot underflow, while new() still rejects N == 0. N == 1 is a valid empty C string. The private field and every construction path form a module-wide validity invariant.

StaticString::as_bytes() excludes the terminator without copying; as_c_str() uses CStr::from_bytes_with_nul_unchecked only because the constructor established its exact invariant; as_c_ptr() returns the thin read-only C pointer. Copying the wrapper duplicates only the pointer, not the bytes. A const fn alone does not establish flash placement: inspect literal storage, the separate wrapper/reference object, linker sections and startup copies. No method is assumed zero-cost until assembly and whole-image measurements prove it.

Benchmark three representations: (A) nonterminated &'static [u8; N], (B) native &'static CStr, and (C) StaticString<N>. Compare total flash, static RAM, startup copies, metadata, generated functions, helper calls, stack and cycles per transmitted byte. Include embedded-NUL behavior, arbitrary non-UTF-8

bytes, empty strings, N == 0 rejection, long-message streaming and the effects of const-generic code duplication. A further custom string type is justified only by a distinct measured requirement.

5. Initial platform and runtime boundaries

Selected exact MCU part numbers: STM8S103F3P6 (lead compiler bring-up) and STM8S003F3P6 (compatibility qualification). User-confirmed device selection does not substitute for verifying physical board markings, clocks, package wiring, and BOM before tests. No currently working Rust target is implied.

- Hardware: run F0 and F1 on STM8S103F3P6, then replicate the F2 qualification suite on STM8S003F3P6. Check physical part markings, board clocks, SWIM/debug access, pin wiring and instrumentation before hardware claims.
- Runtime: no_std, no alloc, panic abort, one execution core; startup/linker work may remain in a C shim initially.
- Language feature contract: Section 3 is the single authoritative support matrix, published as a machine-readable versioned contract. Initial: bool, unit, u8/i8/u16/i16/u32/i32, 16-bit usize/isize, references, arrays, structs, tagged enums, slice references &[T] and &mut [T], ordinary statics, function pointers, const fn, static-dispatch generics, closures/Drop after validation, fixed-size array references &'static [u8; N], and UTF-8/C string literals with &str and &CStr after storage/layout/ABI validation. Experimental: StaticString<N> after checked-construction and whole-image optimization validation. Deferred: char and higher Unicode processing, u64/i64, f32/f64, trait objects, async, unions, packed/aligned layouts, inline assembly and recursion. Excluded initially: alloc::String, format!, u128/i128, allocation, unwinding and threading. static mut remains unsafe-only. Unsupported constructs require clear diagnostics. C bitfields require explicit C shims or accessor functions: repr(C) alone does not define Rust bitfields.
- Peripheral access: audited volatile MMIO wrappers with explicit unsafe boundaries; isolate interrupt sharing and synchronization.
- Builds: pinned rustc nightly/revision, pinned SDCC version, reproducible options, map files, binary-size and stack reports.

Target layering: (1) rustc architecture target fixes STM8 data layout, medium model and calling convention; (2) exact MCU manifest supplies memory map, register set, vectors, package pins, startup/linker details; (3) board configuration supplies crystals/clock, populated components, programming pins and instrumentation. MCU metadata belongs primarily in PAC/HAL and linker generation, not in MIR-to-C lowering. Define Send/Sync conservatively for peripheral tokens and interrupt-shared state; do not claim a safe HAL until aliasing and synchronization have been verified.

6. Migration scenario for existing C firmware

Prefer incremental migration, not a wholesale rewrite. First compile an existing SDCC C firmware unchanged. Replace non-interrupt application modules with Rust-produced C objects, keep the known startup/interrupt and linker configuration, and validate the same functional test cases. A useful first candidate is a sensor state machine, command parser, or protocol encoder: typed logic with limited direct MMIO and a small API boundary.

A representative application should be able to express a small GPIO abstraction using Rust structs, methods, mutable borrowing, const functions and volatile operations; these constructs need not imply a heap. But exposing only Rust syntax while silently generating incompatible C layout or non-reentrant calls would make migration unsafe.

7. Reliability and testing strategy

Layer	Purpose	Evidence to capture
Host semantic differential tests	Compare Rust reference operations with generated C	Boundary-value corpus; equivalent operation outputs
STM8 simulator (uCsim)	Validate emitted instruction behavior and memory	Registers, stack, data memory, call/return traces
Physical MCU tests	Validate silicon-specific behavior	Interrupt timing, peripheral I/O, long-run tests
ABI cross-language tests	Prove mixed C/Rust interoperability	Calls in both directions; aggregates; pointers; ISR shims
Optimization matrix	Detect transformation-induced wrong code	SDCC optimization levels; pinned compiler hashes
Resource budget regressions	Prevent unusable flash/RAM/stack growth	Flash, RAM, stack high-water, runtime benchmarks

Use a compiler regression suite with boundary-value oracles and minimized reproductions. Native Rust on a host is an arithmetic reference only for target-independent semantics, never STM8 endianness, layout, pointer sizes, address spaces or ABI. Capture Rust source, MIR, generated C, assembly, toolchain flags, simulator and silicon results. Exercise SDCC optimization profiles. Stack high-water measurements describe executed paths; combine them with call-graph analysis, interrupt nesting and conservative bounds under the same assumptions for C and Rust.

8. Milestones, staffing, and decision gates

Milestone	Illustrative elapsed time (one engineer)*	Exit gate
Feasibility spike	4–8 weeks	F0 passed on lead STM8S103F3P6; core blocker inventory and resource baseline published.
Embedded prototype	3–6 months	F0 achieved; basic feature subset operational; F1 ABI/ISR risks quantified.
Migration-capable beta	6–12 months	F1 passed: bidirectional ABI, interrupt shim, helper audit and mixed C/Rust application migration on lead MCU.

Compatibility qualification (F2)	After F1; before release	STM8S003F3P6 qualification suite passed with independent device manifest, hardware results and resource reports.
Documented stable subset	12–24+ months (conditional; likely longer if core/ABI prove difficult)	F2 repeated as final qualification, plus release gates and SPL example / resource parity suite.

*Highly uncertain planning scenarios for an experienced compiler engineer, not empirical delivery forecasts. rustc integration and target core support can invalidate these numbers. “Stable subset” means stability of the documented implementation under pinned tools, not stable Rust upstream target status.

8.1 Suggested go/no-go prototype

F0 — Lead compiler feasibility (STM8S103F3P6): demonstrate a pinned custom target with coherent endian/width/alignment/atomic contracts, MIR → C → SDCC → uCsim → physical hardware, checked/unchecked signed/unsigned 8/16/32-bit arithmetic, MIN/-1 divide and remainder handling, shifts, loops, branches, references, arrays, struct, tagged enum, validity tests, and byte-accurate volatile MMIO traces. Attempt build-std=core and classify blockers with a credible remediation plan. Deliver assembly, flash/RAM/stack reports. F0 NO-GO/redesign: incoherent target layout, wrong arithmetic/validity/volatile behavior, no executable hardware pipeline, unresolvable core blockers, or basic-operation expansion over 4x equivalent SDCC C code/stack without identified fix. F0 can pass with ABI work still pending but not with an implausible route to F1. String/storage F0 cases: (1) an ASCII literal retained in flash with no startup RAM copy; (2) &str passed between functions to validate pointer/length representation and ABI; (3) RAM-backed &str from validated bytes; (4) constant string equality using bounded length-aware byte operations; (5) a multibyte UTF-8 literal checking byte length and as_bytes() (for example, Привет is 12 bytes) without Unicode-heavy routines; (6) a long flash-resident UART message streamed with bounded stack; (7) native C literals covering trailing-NUL storage, to_bytes(), to_bytes_with_nul(), as_ptr() C calls, arbitrary non-UTF-8 bytes and compile-time rejection of interior NUL; and (8) a three-way comparison of &'static [u8; N], native &'static CStr and StaticString<N>, including checked const construction, N == 0 rejection, N == 1 empty-string behavior, metadata, monomorphization and whole-image cost. Inspect literal and reference-object placement separately.

Optional AVR reference experiment (time-boxed; not an F0 completion requirement): build a small avr-none program with a pinned toolchain, capture its target specification, MIR, core configuration and required helpers, and run it in a simulator or on hardware. Reuse suitable target-independent semantic cases in the STM8 corpus. AVR layout, ABI, instruction output and resource results are not STM8 oracles. [23]

F1 — Migration feasibility (lead MCU): mandatory both C→Rust and Rust→C calls under measured __sdcccall(1) ABI; tested interrupt entry and priority-preserving nested critical sections; linked-helper reentrancy audit; successful equivalent GPIO/timer/UART sample applications with baseline results. F1

NO-GO: any unresolved ABI or ISR corruption, added/removed/reordered observable MMIO operations, unbounded stack use, sample firmware exceeding 8 KB flash or 1 KB RAM, or basic-operation flash/cycle overhead over 2x against direct-register C under equivalent behavior. These are interim stop thresholds, not final SPL-parity release goals.

F2 — Compatibility qualification: replay lead-target hardware-sensitive and applicable SPL/semantic tests on the STM8S003F3P6 with its independently checked manifest, board setup, pin assignments and memory reports. F2 does not block F1 work; F2 failure blocks claims of STM8S003F3P6 qualification.

Release gates — require the versioned feature contract and explicit unsupported-feature diagnostics; both target-specific qualification results; compiler regression and functional SPL-derived suite; full behavioral equivalence; validated critical sections/reentrancy; reproducibility; separately reported binary/flash, RAM/stack, cycles and interrupt latency; and per-workload resource parity against equivalent SDCC SPL C, with documented release-approved exceptions limited to non-critical timing. F0/F1/F2 are the only staged decision gates.

Timeline convention: milestone times in the table are illustrative cumulative elapsed time from project start for one experienced engineer, not person-month commitments. Completion of broad core support or a stable subset may require substantially longer depending on F0/F1 findings.

9. Usability and performance acceptance criteria

Resource parity is a principal release objective, measured per selected functional workload against equivalent SPL C: no greater occupied flash or static RAM, no regression in conservative stack bound, and no extra execution cycles for timing-critical operations. Non-critical routines may be tracked against a provisional <=5% cycle tolerance only if disclosed explicitly; exceptions require individual documented approval and never count as parity. Aggregate results supplement, not replace, per-workload figures.

The user experience should eventually be Cargo-like, but the initial build system can be a small wrapper that invokes pinned rustc components, the translator, SDCC, and the STM8 linker. Include errors for unsupported features, linker map reports, and a published set of tested MCUs and SDCC versions.

9.1 Why LLVM parity does not automatically transfer to SDCC

Rust/LLVM often competes well with optimized C on mature CPU targets, but LLVM contributes its own middle-end and backend optimizations. Our proposed MIR → C → SDCC route does not inherit LLVM interprocedural optimization, LLVM instruction selection, or LLVM link-time optimization. SDCC supplies mature STM8 code generation, yet poorly lowered C can hide constants, introduce temporaries, inhibit inlining, increase stack usage, or retain unused functions. Therefore efficiency must be measured from the first prototype; “zero-cost abstraction” is a goal to demonstrate, not a guarantee.

9.2 Compile-time target and peripheral specialization

The selected devices STM8S103F3P6 and STM8S003F3P6 have finite documented peripherals, register maps and pins. Translate SPL-style preprocessor selection into versioned MCU manifests, PAC/HAL feature gating, const generics, static dispatch and constant addresses; reject unavailable peripherals at compile time. This happens above the architecture-level rustc target.

Recommended mechanism: select one exact orderable part number through a Cargo feature or explicit project/device configuration; load a versioned, reviewed description derived from ST RM0016, part datasheets and headers; expose device-specific PAC/HAL modules and cfg flags; reject absent peripherals or pins at build time; specialize const-generic register addresses and masks; and remove unreachable monomorphizations before generating C. A generated definition is not trusted until checked against source documents and physical hardware.

Prefer typed per-part generated register definitions over scattered preprocessor conditions: device-feature selection chooses exactly one validated part manifest; constants include MMIO addresses, register widths and reset values, interrupt vectors, pin availability, and peripheral presence. Compile-time checks reject unavailable peripherals and contradictory feature combinations. Rust const generics and monomorphization specialize accessors to fixed addresses and masks, while reachability pruning discards unreferenced code.

Do not claim all peripheral accesses are zero-cost: compare actual SDCC assembly for volatile read/modify/write sequences, atomicity and side effects. Device manifests must be checked against ST RM0016 and exact-part datasheets and versioned independently of the compiler.

Example Rust API (illustrative, not a compiler claim):

```
/// UNSAFE code-generation illustration, NOT a safe HAL API.
/// Arbitrary MMIO addresses/masks may be invalid and multiple pins alias a port.
unsafe fn set_bits_unchecked(odr: *mut u8, mask: u8) {
    let v = odr.read_volatile();
    odr.write_volatile(v | mask);
}
// A safe HAL would need validated, non-forgeable device tokens,
// port-wide ownership/synchronization, restricted constructors,
// and an interrupt-safe, audited read-modify-write strategy.
```

This only illustrates the lower-level code-generation pattern. A safe HAL must reject arbitrary addresses/masks, enforce ownership across a shared port, implement interrupt-safe read-modify-write, and audit generated assembly. An &mut self receiver by itself does not prevent register aliasing or ISR races.

Type-state wrapper example (illustrative API, not a safe-HAL proof): marker types and PhantomData carry no runtime state. Consuming self in into_output prevents continued use of the old input token. Construction must supply validated, non-forgeable registers and ownership; configure_output() must perform the required DDR/CR1/CR2 writes before returning Output. Shared-port and ISR synchronization remain mandatory.

```
use core::marker::PhantomData;

struct Input;
struct Output;

struct GpioRegisters;

struct GpioPin<State> {
    registers: *mut GpioRegisters,
    _state: PhantomData<State>,
}

impl GpioPin<Input> {
    fn into_output(self) -> GpioPin<Output> {
        unsafe { configure_output(self.registers); }

        GpioPin {
            registers: self.registers,
            _state: PhantomData,
        }
    }
}

impl GpioPin<Output> {
    fn set_high(&mut self) {
        unsafe { write_odr_high(self.registers); }
    }
}
```

The marker states disappear after monomorphization; only the required MMIO remains. This is a compile-time interface check, not evidence of safe addresses, port ownership or ISR correctness. A const-generic BASE/MASK variant may make the token zero-sized, but constructors must still reject arbitrary addresses, masks and states.

Treat reviewed per-device definitions as a versioned PAC/HAL and linker-generation input: include memory/flash limits, EEPROM, pinout and alternate functions, register addresses, vectors and supported peripherals. Test compile-time rejection for unavailable pins/features. Match generated symbol/object dead stripping to SDCC limitations; benchmark whole-image effects, not only isolated methods. Never silently assume two similarly named MCU variants are identical.

9.3 Optimization policy for an 8 KB flash / 1 KB RAM class

For representative STM8S003F3/STM8S103F3 configurations, flash is 8 KB and SRAM 1 KB. At these scales even hundreds of excess bytes can make a migrated application unusable. Optimize total firmware, not just isolated functions. The backend should preserve constant operands, perform dead-code elimination after monomorphization, select carefully when to inline, minimize copies and helper calls, lower volatile MMIO directly, and organize C output/linkage so unused functions and peripherals do not survive linking. Aggressive inlining can increase flash size; size-versus-speed choices must be explicit and benchmarked.

String benchmarks must compare equivalent C and Rust static messages and UART streams, prioritizing the three-way &'static [u8; N], &'static CStr and StaticString<N> matrix. Record literal-byte and reference-object placement, startup copies, occupied flash, static RAM, stack, generated functions, helper calls and cycles per transmitted byte. Include embedded-NUL behavior for &str, trailing-NUL behavior for &CStr and StaticString<N>, arbitrary display-controller bytes, empty strings and the

trade-off between thin compile-time array references and const-generic code duplication. Any claimed zero-cost abstraction requires equivalent observable behavior and assembly/resource evidence.

9.4 Required baseline: SPL C, direct-register C, and Rust

Reference baseline A: official ST STSW-STM8069 SPL sources ported and validated for SDCC with a pinned third-party adaptation (such as STM8-SPL-SDCC), its commit and patch set. The adaptation states its examples are work in progress: no example is presumed to build or run unchanged. Compare (A) C application + C SPL, (B) Rust application + C SPL (migration/ABI), and (C) Rust application + Rust PAC/HAL/drivers (native driver coverage). Add direct-register C for a microbenchmark comparison and optional IAR/Cosmic optimized C reference builds. Capture exact SPL source, compiler, startup/linker configuration, file splitting, flags, source modifications, input stimuli and observable behavioral contracts.

Lead STM8S103F3P6 must participate in initial hardware runs; repeat the supported matrix on STM8S003F3P6 before qualifying the compatibility target. Capture C and Rust generated assembly, image/map bytes, stack high-water evidence, static call-graph estimates and cycle counts.

9.5 Revised performance acceptance milestones

Resource and parity policy (single authoritative definition): each test must report occupied flash address bytes (vectors, startup, code, constants, initialization images, helpers; not HEX text length); static data RAM as .data + .bss, excluding stack; separately reserved stack and other allocations; total RAM as static data + reserved stack + other reservations; measured stack high-water and conservative call-graph/ISR-nesting bound calculated under equal assumptions; calibrated hardware cycles (measurement overhead reported); and ISR latency from a defined event to a defined handler action, including entry shim. Evaluate each named benchmark individually; aggregates may be published but never conceal a failing per-workload resource limit. Define complete behavioral equivalence, including SPL parameter checks, configuration, error handling and side effects. Direct-register shortcuts lacking such behavior must be labeled non-equivalent or normalized. Release goal: on each equivalent SPL workload, no larger occupied flash or static RAM and no slower execution; require conservative stack bound <= equivalent SPL baseline, using identical analysis assumptions and interrupt nesting. Noncritical speed deviations up to 5% may be proposed as documented exceptions only when hardware budgets and release approval permit; hard silicon resource limits cannot be waived.

These parity targets are hypotheses and release criteria, not predicted performance. If the MIR → C representation consistently loses against direct-register C even on isolated constant-register operations, stop feature expansion and improve lowering/optimization before investing in broad core compatibility.

9.6 SPL example-driven completeness and validation

Build an example inventory from the STSW-STM8069 SPL examples, filtered against peripheral availability and pin/package capability for STM8S103F3P6 and STM8S003F3P6. These are candidate tests, NOT assumed ready-to-run SDCC examples: first explicitly port each example to the pinned SDCC SPL adaptation, validate its C hardware baseline, and archive stimulus and observable behavior. Then collect (A) C application + C SPL, (B) Rust application + C SPL, and (C) Rust application + Rust

PAC/HAL/drivers. Only configuration (C) counts as native Rust peripheral-driver coverage. Hardware tests shall use SWIM/debugger where available, breakpoints/register/memory observations, oscilloscope or logic analyzer for pin waveforms, UART/SPI/I2C traffic and interrupt timing, calibrated timers for cycles, and repeatability/error budgets. uCsim is supplementary; its cycle timing must be validated before treating simulator timings as measurements.

Machine-readable test matrix: exact MCU and board revision, supported peripheral, SPL example and adaptation patch, functional contract and stimuli, toolchain name/version, rustc revision, MIR-to-C backend revision, Rust test-case revision, MCU manifest revision, board configuration, optimization profile, flags, disassembly source/method, binary/map artifacts, flash address occupancy, .data/.bss, stack allowance/high-water/analysis, hardware cycle and IRQ latency definitions, simulator result, silicon result and pass/fail/blocked reason. Preserve generated C and MIR for regression. Disassembly and binary identity are diagnostic, not correctness oracles; independently compare observable behavior.

9.7 Commercial compiler comparison and reference corpus

Optional commercial optimization references: Cosmic STM8 Free Special Edition is advertised without code-size restriction; registration and a renewable one-year license are required, and the license permits commercial use. Its experimental Linux STM8/Eclipse environment targets Ubuntu 20.04 and needs current validation. IAR offers a 14-day evaluation with compiler and debugger, but STM8-specific Linux availability, object/listing output and exact optimization option syntax must be verified using the installed distribution. Prepare project/automation before evaluation activation; batch compilation of selected examples across debug/none, low, medium, high/balanced, high/size and high/speed is expected to fit comfortably in the trial duration, but environment setup and example adaptation may take longer. Archive build logs, map, object (where available), linked executable, HEX/BIN and disassembly; verify symbol and debug-info availability instead of assuming it for every build. Neither proprietary compiler is a dependency or semantic oracle. Compare controlled common configurations and separately best-tuned per-toolchain results. Prepare a reproducible SPL-example matrix and build scripts before initiating time-limited commercial compiler evaluations. Enumerate debug/unoptimized, low, medium, high/balanced, size-optimized and speed-optimized profiles using each compiler's actual supported flags. A single batch should capture compiler/assembler/linker logs, maps, source/objects (where available), firmware images and independent disassembly, all associated with source and toolchain hashes. Batch execution is expected to be short relative to a 14-day trial once the environment works; no exact one-hour guarantee is made. Build collection precedes functional validation: on physical boards, use SWIM debugging and oscilloscope/logic analyzer measurements and archive traces. Commercial binaries are comparative evidence rather than byte-for-byte targets.

10. Principal risks and mitigations

Risk	Impact	Mitigation
rustc internals and backend API churn	High maintenance cost	Pin toolchain; isolate rustc integration; maintain compatibility CI

Missing core/compiler-builtins support	Blocks idiomatic no_std	Separate core bootstrap milestone; selective intrinsic inventory
C undefined behavior introduced by lowering	Silent wrong code	Semantic lowering rules and differential fuzz/property tests
Memory-space or ABI mismatch	Corruption, bad calls	One lead device first, then separate compatibility qualification; exhaustive layout/interop tests
Poor code quality/stack bloat	Firmware no longer fits	Benchmark against C; optimize lowering patterns; inspect SDCC assembly
Toolchain errors hidden by generated C	Difficult debugging	Source maps/line directives, dumps and reproducible test cases

10.1 Open-source licensing and third-party artifacts

Project results are intended for a public GitHub repository, with no specific commercial plans. Project-owned compiler code, build tooling, documentation, and original tests will be dual-licensed under MIT and Apache-2.0 at the recipient's option. This licensing applies only to original project work.

- Third-party components: ST SPL sources, SDCC adaptations, libraries, headers, and other dependencies retain their original licenses and will not be relicensed. Track origin, revision, license, and local modifications in THIRD_PARTY.md; prefer download scripts, patches, or submodules where appropriate.
- SDCC and runtime outputs: using SDCC does not by itself relicense original Rust or generated C source. Check separate runtime-library notices for linked objects before distributing firmware. Do not publish proprietary installers, activation data, license files, or restricted runtime sources.
- Commercial references: Cosmic FSE may be used under its renewable license; IAR is limited to the permitted evaluation and comparison, not ongoing product development. Publication of commercial-compiler binaries or disassembly is subject to the installed license terms, especially when proprietary runtime code is linked.
- Public CI and hardware: reproducible mainline builds use only open rustc/SDCC tooling and may run on public GitHub-hosted runners. Proprietary collection is a separate manual activity; do not expose a licensed self-hosted Windows machine to untrusted pull requests. Keep hardware-in-the-loop testing local or carefully controlled.

11. Overall assessment

Feasibility: credible for selected MCUs, a restricted runtime, and a documented set of Rust features. Complexity: high, mostly in target integration, semantic lowering, optimization and verification—not parsing Rust. Usability: potentially good for migrating simple C projects incrementally if SPL parity is achieved. Production confidence: attainable only after extensive optimization-aware testing and on-hardware regression evidence.

The recommended project is a research-backed, Rust-compatible no_std STM8 firmware toolchain using SDCC code generation. Its first deliverable should be a feasibility demonstrator with clear blockers and measured output, not a promise of “full Rust for STM8.”

12. References and further reading

[1] SDCC manual — https://sdcc.sourceforge.net/doc/sdccman.pdf

[2] ST STSW-STM8069 SPL — https://www.st.com/en/embedded-software/stsw-stm8069.html

[3] ST RM0016 — https://www.st.com/resource/en/reference_manual/cd00190271.pdf

[4] ST PM0044 CPU programming manual — https://www.st.com/resource/en/programming_manual/cd00161709.pdf

[5] STM8S103F3 device — https://www.st.com/en/microcontrollers-microprocessors/stm8s103f3.html

[6] STM8S003F3 device — https://www.st.com/en/microcontrollers-microprocessors/stm8s003f3.html

[7] Third-party STM8-SPL-SDCC adaptation — https://github.com/bschwand/STM8-SPL-SDCC

[8] SDCC bug #3500 — https://sourceforge.net/p/sdcc/bugs/3500/

[9] rustc custom targets — https://doc.rust-lang.org/rustc/targets/custom.html

[10] Cargo build-std — https://doc.rust-lang.org/cargo/reference/unstable.html#build-std

[11] Rust type layout — https://doc.rust-lang.org/reference/type-layout.html

[12] Rust arithmetic overflow — https://doc.rust-lang.org/reference/expressions/operator-expr.html#overflow

[13] rustc code generation guide — https://rustc-dev-guide.rust-lang.org/backend/codegen.html

[14] Cosmic experimental Linux STM8 package — https://license.cosmic.fr/download_stm8_eclipse.php

[15] IAR evaluation — https://www.iar.com/embedded-development-tools/free-trials

[16] IAR Embedded Workbench IDE guide (historical generic reference) — https://wwwfiles.iar.com/stm8/guides/EW_IDEGuide.pdf

[17] SDCC official homepage and release news — https://sdcc.sourceforge.net/

[18] SDCC 4.6.0 release plan and mature target list — https://sourceforge.net/p/sdcc/wiki/SDCC%204.6.0%20Release%20Plan/

[19] rustc dynamically loadable codegen backend interface (unstable) — https://doc.rust-lang.org/unstable-book/compiler-flags/codegen-backend.html

[20] rustc backend-agnostic codegen — https://rustc-dev-guide.rust-lang.org/backend/backend-agnostic.html

[21] Rust target tier policy — https://doc.rust-lang.org/rustc/target-tier-policy.html

[22] Cosmic STM8 free edition FAQ — https://cosmicsoftware.com/faq/faq50.php

[23] Rust avr-none target documentation — https://doc.rust-lang.org/rustc/platform-support/avr-none.html

[24] Current rustc AVR target specification — https://github.com/rust-lang/rust/blob/master/compiler/rustc_target/src/spec/targets/avr_none.rs

[25] Rahix AVR-HAL embedded ecosystem — https://github.com/Rahix/avr-hal

[26] rust-embedded critical-section API — https://github.com/rust-embedded/critical-section

[27] AVR-Rust Guidebook (historical reference) — https://github.com/avr-rust/book.avr-rust.com

[28] Experimental rustc_codegen_c MIR-to-C backend — https://github.com/rustic-compiler/rustc_codegen_c

[29] Rust core string-slice module — https://doc.rust-lang.org/core/str/index.html

[30] Rust alloc::String owned heap string — https://doc.rust-lang.org/alloc/string/struct.String.html

[31] AVR-Progmem utilities and PmString — https://github.com/Cryptjar/avr-progmem-rs

[32] Rust core::ffi::CStr — https://doc.rust-lang.org/core/ffi/struct.CStr.html

[33] Rust C-string literals — https://doc.rust-lang.org/edition-guide/rust-2021/c-string-literals.html

Source note: This is an F0 baseline specification, not evidence that the proposed toolchain already works. Engineering thresholds require experimental validation on the selected hardware. Current compiler options and artifact availability must be checked against the installed versions.